

VECTOR DATABASES

Production • Local • POC • Trade-offs • Enterprise GCP perspective

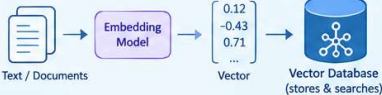


Vector Databases – Types, Examples, Use Cases & Trade-offs

Different types of vector databases serve different needs. Here's a clear comparison to help you choose the right one for your use case.

What is a Vector Database?

A dedicated vector database is a database whose architecture is specifically designed around storing and searching vectors.



The 4 Types of Vector Databases



Dedicated

Purpose-built for vector storage and search.



DB Extensions

Adds vector capability to existing databases.



Embedded

Local, lightweight libraries/stores.



Managed Cloud

Fully managed, no infrastructure management.

Type	Examples	Origin / Type	Typical Strength	Key Features	Best For	Notes
 Dedicated Vector Database Built specifically for vector storage, indexing and search.	 Pinecone  Qdrant  Weaviate  Milvus	Pinecone – Commercial (managed) Qdrant – Open-source Weaviate – Open-source Milvus – Open-source	Simplicity + managed scale Filtering + developer flexibility Vector + hybrid/multimodal capabilities Very large-scale vector workloads	- Specialized vector indexes (HNSW, IVF, PQ) - Fast similarity search (ANN) - Metadata filtering - Scalable & distributed architecture - APIs, security, monitoring	High-scale vector workloads	Notes: - Fastest ANN performance - More operational complexity (for OSS) - Vendor lock-in risk (for managed)
 DB Extensions Adds vector search to existing databases.	 pgvector  Redis  Elasticsearch	PostgreSQL extension In-memory DB Search engine	Simplest integration Low latency Full-text + vector search	- SQL support (pgvector) - Transactions & joins - Simple deployment - Integrates with existing tools	Existing DB setups	Notes: - Easiest to adopt if you already use these systems. - May have limited vector-specific features.
 Embedded Lightweight, local vector stores/libraries.	 FAISS  Chroma  Annoy	Meta AI library Open-source Open-source	Very fast similarity search Developer friendly Simple & lightweight	- No server needed - Runs locally (CPU/GPU) - Easy to integrate (Python, etc.) - Good for prototyping / small scale	Local/edge, small datasets	Notes: - Not suitable for very large-scale production. - Limited enterprise features (HA, security, etc.).
 Managed Cloud Fully managed vector services on cloud platforms.	 Pinecone Serverless  Qdrant Cloud	Commercial (managed) Open-source (managed)	Production ready Flexible & scalable	- Zero infrastructure management - Auto-scaling - High availability - Security & compliance	Production RAG at scale	Notes: - Higher cost - Less control over infrastructure - Vendor dependency

When to Use What?

Dedicated
Choose when you need high performance, scale, and advanced vector search capabilities.

DB Extensions
Choose when you want vector search with your existing database and minimal changes.

Embedded
Choose for local development, experimentation, or small use cases.

Managed Cloud
Choose for production workloads where you want zero ops and can accept higher cost.

Key Trade-offs

Factor	Dedicated	DB Extension	Embedded	Managed Cloud
Performance	★★★★	★★	★★	★★★★
Scalability	★★★★	★★	★	★★★★
Ease of Setup	★★	★★	★★★★	★★
Operational Overhead	★★	★★	★	★★★★
Cost	★★	★★	★★★★	★★
Control / Flexibility	★★★★	★★	★★	★★

Remember

- ✓ Dedicated ≠ Managed. (e.g., Pinecone is both). Vector DBs are not just "a database with vectors."
- ✓ The right choice depends on your scale, budget, existing infrastructure and use case.
- ✓ In your GCP environment, also evaluate # Vertex AI Vector Search alongside external options (Pinecone, Qdrant, Weaviate, Milvus).

What is a vector DB? Stores embeddings and retrieves vectors close to a query vector; usually keeps ID, text/metadata and vector indexes.

How it is formed Document → parse → chunk → embedding model → vector + metadata/ACL → HNSW/IVF/vendor index → searchable collection.

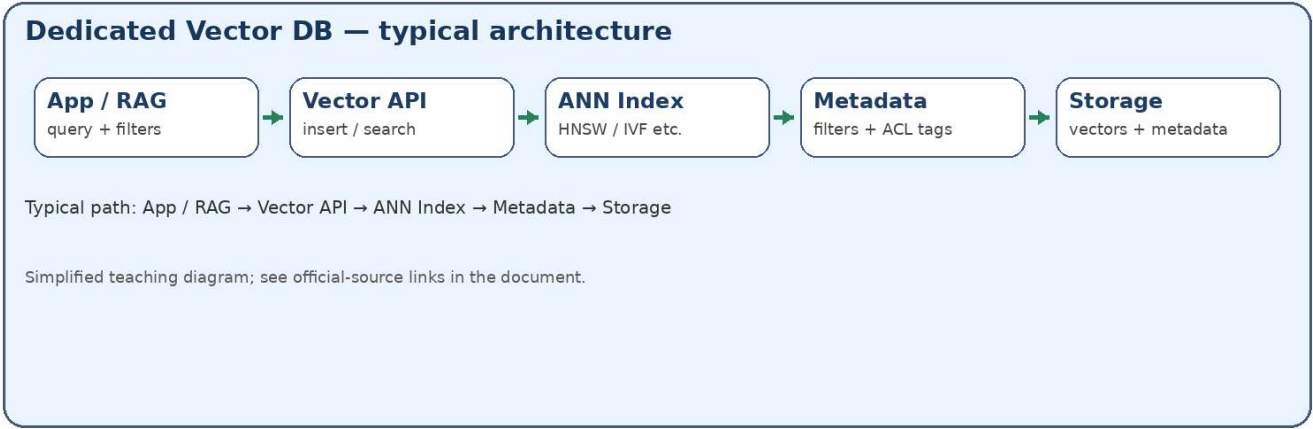
Core distinction Dedicated / Extension / Embedded describe the technology; Managed Cloud describes the operating model. A product can therefore appear in more than one category.

Index 2 Dedicated • 3 DB Extensions • 4 Embedded • 5 Managed Cloud • 6 Decision guide + GCP + sources

Vector Database Research • 1

2. Dedicated Vector Databases

Purpose-built around vector storage, ANN/vector indexing and similarity retrieval.

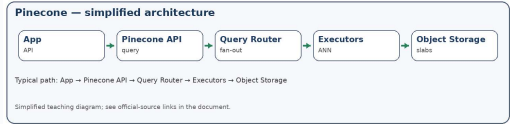


Simplified architecture based on official vendor documentation.

Dedicated examples

Mini diagrams show the core idea, not every internal component.

Pinecone



Profile: Pinecone Systems; commercial/proprietary managed. Documented user: Gong Smart Trackers; customers include Adobe, Microsoft, OpenAI.

Strengths: Managed/serverless scale; metadata filtering; distributed query execution; low ops.

Trade-offs: Recurring cost; vendor dependency; less infra control.

Fit: Production RAG/search/recommendations when managed ops are preferred.

Weaviate



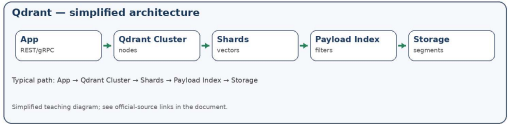
Profile: Weaviate B.V.; BSD 3-Clause open source + Cloud. Booking.com selected it as a vector DB standard.

Strengths: Vector + BM25/hybrid; HNSW; filtering; multimodal modules; scaling.

Trade-offs: More features/components to operate; self-hosted sizing/HA.

Fit: Enterprise RAG/search needing hybrid retrieval and multimodal data.

Qdrant



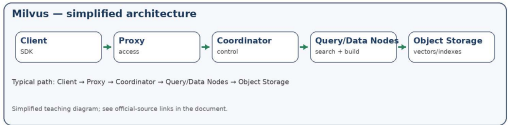
Profile: Qdrant; Apache 2.0 open source + Cloud. Customers include HubSpot, Tripadvisor, Dailymotion and enterprise/legal users.

Strengths: Payload filtering; sharding/replication; Rust; self-host or cloud.

Trade-offs: Self-hosting needs cluster/HA engineering.

Fit: Production RAG, hybrid/multimodal retrieval and filtering-heavy workloads.

Milvus



Profile: Developed by Zilliz; Apache 2.0 open source; Zilliz Cloud managed. Adopters include eBay/HP.

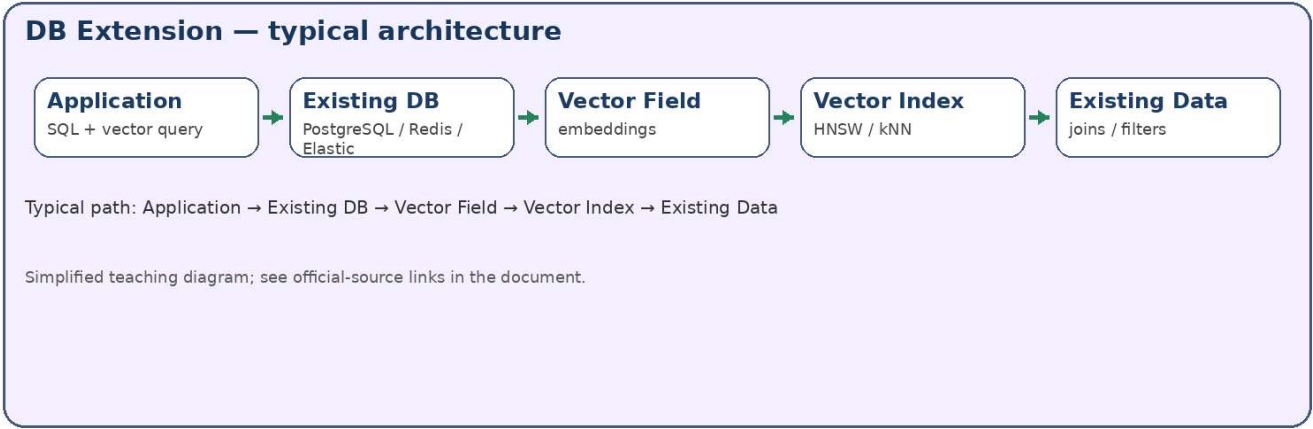
Strengths: Massive-scale design; distributed/cloud-native; separate query/data/streaming workers; many indexes.

Trade-offs: Distributed architecture is powerful but operationally complex.

Fit: Very large vector platforms, recommendations and deep infra control.

3. Database Extensions / Vector-capable General-Purpose Databases

Add vector storage/search to a database or search engine that already handles structured data, transactions, filters or full-text search.

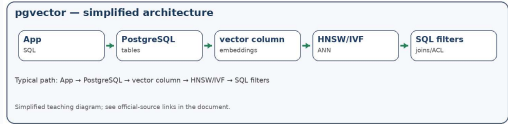


Simplified architecture based on official vendor documentation.

Extension examples

Mental model: “Can we add vector retrieval to a strategic database/search platform instead of introducing another system?”

pgvector



Profile: PostgreSQL extension; open source, PostgreSQL license. Used by Supabase's Postgres AI/vector offering.
Strengths: SQL + vectors; ACID/JOINS; HNSW/IVFlat; existing Postgres ecosystem.
Trade-offs: At very large scale, specialized vector architecture may be stronger; tuning still matters.
Fit: POC → production when PostgreSQL is strategic.

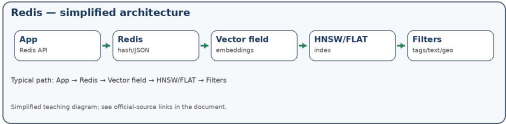
Elasticsearch



Profile: Elastic; licensing includes AGPLv3/SSPL/Elastic options. AI/vector users include Relativity, Docusign, Tavily.
Strengths: BM25 + vector hybrid search; filters; aggregations; security; distributed search.
Trade-offs: More platform complexity; licensing needs review.
Fit: Enterprise search/RAG where lexical + semantic retrieval is central.

Why this category matters It can reduce integration and data-governance complexity because vectors, business data, filters and existing operational tooling stay in one platform.

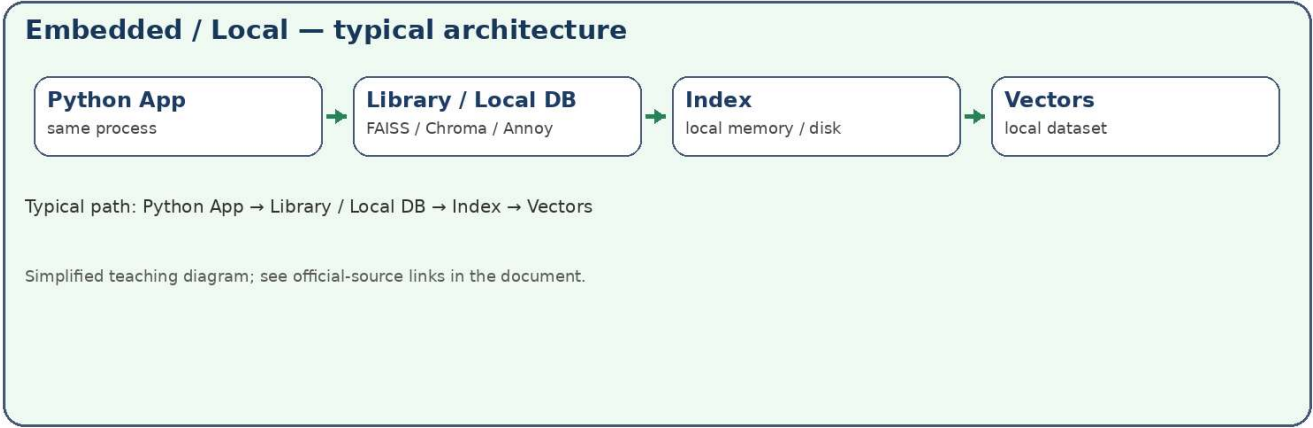
Redis



Profile: Redis Ltd.; current Redis Open Source v8 uses RSALv2/SSPLv1/AGPLv3; commercial products/cloud also exist. Customers include Character.AI/CP AXTRA.
Strengths: Very low latency; vector + text/tag/numeric/geo filters; HNSW/FLAT.
Trade-offs: Memory/cost and licensing/edition considerations.
Fit: Real-time recommendations, semantic search, agent memory, low-latency RAG.

4. Embedded / Local Vector Libraries and Stores

Lightweight local technologies for learning, development, POCs and edge/static workloads; not automatically equivalent to a production distributed database.



Simplified architecture based on official vendor documentation.

Embedded examples

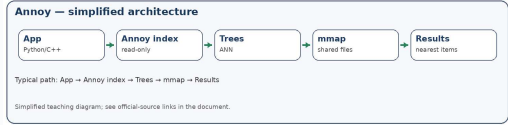
Key distinction: FAISS/Annoy are libraries; Chroma is a lightweight vector store/database with local and service modes.

FAISS



Profile: Meta AI Research; MIT licensed open-source similarity-search library; not a complete distributed DB.
Strengths: Excellent algorithms; CPU/GPU; many index types; strong benchmarking/custom retrieval.
Trade-offs: You must add persistence, APIs, metadata/ACL, HA, replication and ops.
Fit: Local experiments, offline retrieval, benchmarking, custom engines.

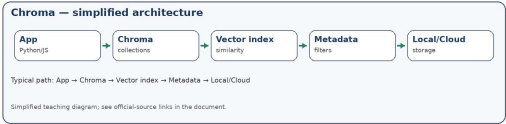
Annoy



Profile: Spotify open-source project; Apache 2.0; C++ library with Python bindings; read-only file indexes.
Strengths: Simple, memory-efficient; mmap sharing; easy local deployment.
Trade-offs: Static/read-only orientation; limited DB features; no native enterprise ACL/HA.
Fit: Small/medium static datasets, edge search and lightweight POCs.

Common mistake “No server needed” means it can run locally/embedded; it does not mean it cannot be exposed as a service. Production suitability depends on HA, persistence, security, observability and scale.

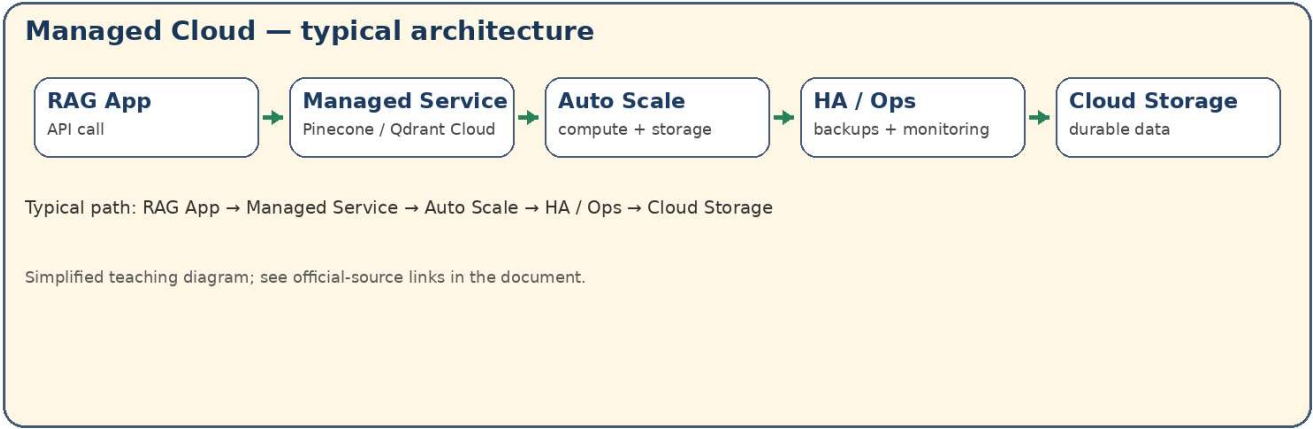
Chroma



Profile: Chroma; Apache 2.0 open source. Local, single-node and distributed; Cloud is managed. Users include Mintlify/Propel/Factory.
Strengths: Developer-friendly collections; vector + full-text + metadata search; smooth local start.
Trade-offs: Local mode is not hardened distributed production; cloud/distributed adds cost/ops.
Fit: RAG prototypes, agent memory, code search and development.

5. Managed Cloud Vector Services

An operating model: the vendor runs infrastructure, scaling, upgrades and much of availability/operations. It overlaps with Dedicated DBs.



Simplified architecture based on official vendor documentation.

Managed examples

Managed Cloud is not a separate technology family in the same sense as Dedicated/Extension/Embedded.

Pinecone Serverless



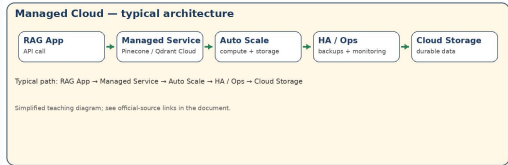
Profile: Pinecone commercial managed service; object storage + elastic query compute. Gong is a documented billion-vector user.

Strengths: Very low ops; elastic scaling; automatic index choices; metadata filtering.

Trade-offs: Recurring cost; vendor dependency; data/network governance.

Fit: Production RAG/search with variable traffic and low-ops priority.

Managed vs self-hosted



Profile: Managed = vendor runs infra. Self-hosted = your team runs servers/Kubernetes, upgrades, backups, monitoring and HA.

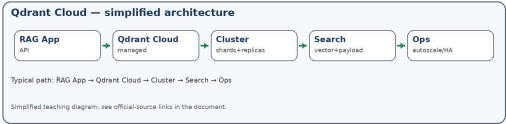
Strengths: Managed usually improves time-to-market; self-hosting can provide more control.

Trade-offs: Neither is automatically cheaper/faster; TCO depends on workload.

Fit: Evaluate TCO, security, residency, SLO/HA, skills and exit strategy.

Overlap to remember Pinecone and Qdrant can be both “Dedicated Vector DB” and “Managed Cloud” because one label describes the technology and the other describes how it is operated.

Qdrant Cloud



Profile: Qdrant managed service based on open-source Qdrant; customer stories include HubSpot/Tripadvisor/Dailymotion.

Strengths: Managed clusters; replication/rebalancing options; familiar Qdrant model.

Trade-offs: Cloud cost; provider dependency; cloud/self-host feature differences.

Fit: Production RAG where the team wants Qdrant without operating the cluster.

6. Decision Guide — Production / Local / POC / GCP

Need	Starting choices	Why	Trade-off	Model
Local learning / tiny POC	FAISS / Chroma / Annoy	Fast, cheap, local	Build prod controls	Embedded
Existing PostgreSQL	pgvector	SQL + vectors together	Scale/tuning may matter	Existing DB
Real-time low latency	Redis	Operational + vector search	Memory/cost/licensing	DB + vector
Enterprise hybrid search	Elasticsearch	BM25 + vector + filters + security	Platform complexity	Search platform
High-scale vector platform	Milvus / Qdrant / Weaviate	Purpose-built scale/control	Ops if self-hosted	Dedicated
Low-ops production	Pinecone / Qdrant Cloud	Vendor handles infra	Cost/vendor dependency	Managed cloud
Your GCP / Gemini Enterprise Agent Platform context Because SharePoint and Confluence are external source systems, evaluate ACL/entitlement filtering, metadata filters, hybrid search, freshness, data residency, network path, observability, HA/DR, cost and vendor lock-in — not only ANN speed.				

Recommended enterprise benchmark

Compare one Google-native option (e.g., Vertex AI Vector Search where applicable) with one/two external dedicated options such as Qdrant/Pinecone/Weaviate/Milvus, and an existing-platform option such as pgvector or Elasticsearch if already strategic. Use the same corpus, embeddings, ACL rules and queries; measure recall@K, p50/p95 latency, freshness, filtered-search quality, availability, cost and operational effort.

Sources researched

- [1] Pinecone architecture: <https://www.pinecone.io/how-pinecone-works/>
- [2] Qdrant architecture/data: https://qdrant.tech/documentation/scaling/distributed_deployment/
- [3] Weaviate architecture: <https://docs.weaviate.io/weaviate/concepts>
- [4] Milvus architecture: https://milvus.io/docs/architecture_overview.md
- [5] pgvector: <https://github.com/pgvector/pgvector>
- [6] Redis vector search: <https://redis.io/docs/latest/develop/ai/search-and-query/vectors/>
- [7] Elasticsearch vector search: <https://www.elastic.co/docs/solutions/search/vector>
- [8] FAISS: <https://github.com/facebookresearch/faiss>
- [9] Chroma architecture: <https://docs.trychroma.com/reference/architecture/overview>
- [10] Annoy: <https://github.com/spotify/annoy>
- [11] Pinecone customers: <https://www.pinecone.io/customers/>
- [12] Qdrant customers: <https://qdrant.tech/customers/>
- [13] Booking.com + Weaviate: <https://weaviate.io/case-studies/booking>
- [14] Milvus use cases: <https://milvus.io/use-cases>
- [15] Chroma customers: <https://www.trychroma.com/updates/customers>
- [16] Supabase + pgvector: <https://supabase.com/docs/guides/database/extensions/pgvector>
- [17] Redis licensing: <https://redis.io/legal/licenses/>
- [18] Elastic licensing: <https://www.elastic.co/pricing/faq/licensing>

Product capabilities, licensing and architectures change; verify current vendor terms before procurement.